

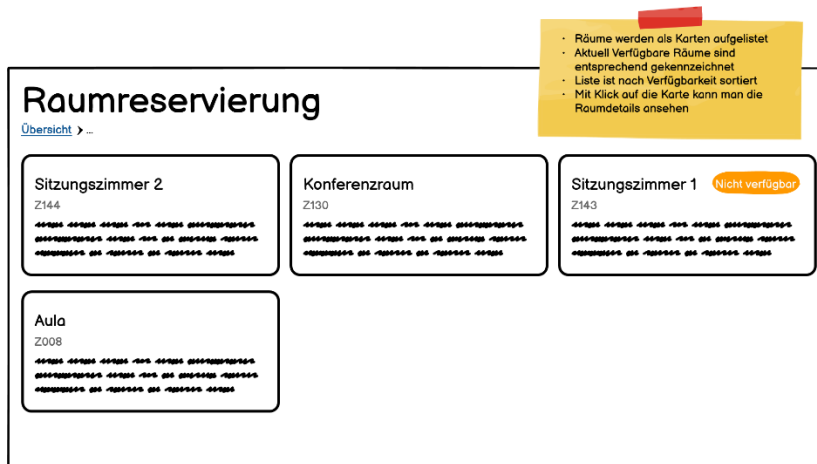
Abschlussbericht SWI1

Applikation	2
Hauptmenu	2
Raumübersicht	4
Reservierung	7
Entwicklung	9
Architektur	9
Codestyle	10
Probleme	10
Stärken	11
Schwächen	11
Reflexion	11

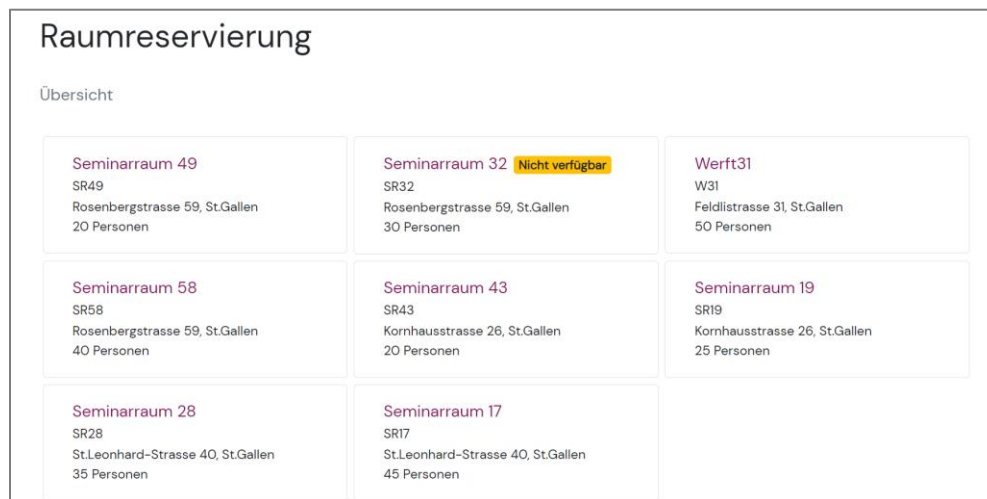
Applikation

Ich habe die Erfahrung gemacht, dass es sich lohnt, umfangreiche Skizzen und Mockups vor der Implementierung zu erstellen. Die Mockups sind im Projekt-Verzeichnis im Ordner *mockups* abgelegt. Folgend einige Gegenüberstellungen von Plan und Produkt.

Hauptmenu



Mockup Raumübersicht



Screenshot Raumübersicht

Auf allen Seiten meiner Anwendung ist zuoberst der Programmname dargestellt. Unterhalb dieses Titels bietet die Breadcrumb Navigationshilfe. Die Breadcrumb ist bewusst prominent in Szene gesetzt, da sie das einzige Navigationselement zwischen den Seiten darstellt. Gleichzeitig fungiert die Breadcrumb als Seiten-Titel.

Das Styling meiner Raumreservierungssystem habe ich dem hübschen OST-Design angeglichen. Ich habe eine spezielle Schriftart von Google-Fonts eingebunden. Tatsächlich gefällt mir diese gar nicht und ich hätte den Standard belassen, wenn die Schrift nicht ausdrücklich gefordert würde. Meiner Meinung nach ist es in den meisten Fällen ein Anti-Pattern, verschnörkelte Schriftarten einzubauen.

Das Hauptmenü stellt alle Räume dar, die von der Rest-Schnittstelle geladen werden. Als UI-Komponente für die Visualisierung habe ich mich für Karten entschieden. Karten sind nicht so langweilig wie beispielsweise eine Tabelle oder Listenelemente. Gleichzeitig ist ein responsives UI mit Karten einfach realisierbar.

Die einzelnen Karten weisen nur die wichtigsten Informationen pro Raum auf. Nur so kann Übersichtlichkeit gewährleistet werden. Diese Abstraktion legitimiert erst den Einsatz einer Übersicht. Auf den Karten habe ich als Hover-Effekt eine Shake-Animation eingebaut. Über den Nutzen dieser Animation kann man sich streiten. Ich finde es gibt dem Ganzen einen verspielteren Touch und animiert zum aktiven Benutzen der Übersicht. Räume die nicht verfügbar sind werden mit einem Badge gekennzeichnet. Da nach Best Practice mit Badges nur Ausnahmefälle hervorgehoben werden, habe ich darauf verzichtet, verfügbare Räume analog zu kennzeichnen. Solche kleine Verbesserungen kumulieren sich und erhöhen die Übersichtlichkeit der Seite.

Es wurden nicht alle Features gemäss Mockup implementiert. So werden die Räume beispielsweise nach Raum-Id und nicht nach Verfügbarkeit sortiert, da verfügbare Räume der Normalfall sind.

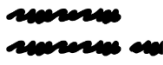
Durch Klicken auf die Karte wird die Raumübersicht geladen.

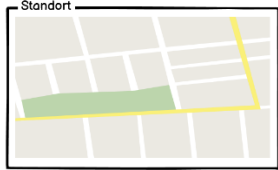
Raumübersicht

Raumreservierung

[Übersicht](#) > Sitzungszimmer 1

Rauminformationen

Raumname: Sitzungszimmer 1
Raum-Id: Z143



Standort


Betrachtungszeitraum: / 

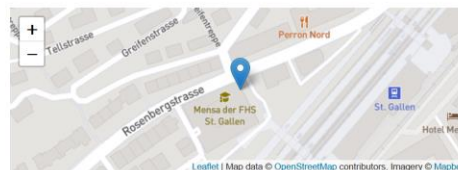

Name	E-Mail	Titel	Von	Bis	
Jonas	jonas.vogel@ost.ch	Präsentation BATH	23. Januar 2021 13:00	23. Januar 2021 14:00	 Löschen
Jonas	jonas.vogel@ost.ch	Coaching PPR3	16. Januar 2021 10:30	16. Januar 2021 12:00	 Löschen

Mockup Raumübersicht

Raumreservierung

[Übersicht](#) / Seminarraum 49 [Reservieren](#)

Name: Seminarraum 49
Kurzname: SR49
Status: Verfügbar
Adresse: Rosenbergstrasse 59, St.Gallen
Kapazität: 20 Personen
Preis pro Stunde: CHF 65.2



Betrachtungszeitraum: 29. December 2020 - 07. February 2021

Organisator	E-Mail	Titel	Von	Bis	
Jonas	jonas.vogel@ost.ch		03. January 2021 19:00	03. January 2021 22:00	 Löschen
Jonas	jonas.vogel@ost.ch	SWI (Best Modul eva)	04. January 2021 13:15	04. January 2021 15:00	 Löschen
Jonas	jonas.vogel@ost.ch	Präsentation BATH	06. January 2021 14:00	06. January 2021 14:30	 Löschen
Jonas	jonas.vogel@ost.ch	Coaching PPR3	28. January 2021 19:00	28. January 2021 20:00	 Löschen

Screenshot Raumübersicht

Die Raumübersicht war die aufwendigste Seite meiner Applikation. Die Raumübersicht besteht aus einem Panel mit Rauminformationen, einem Panel mit der visuellen Darstellung des Standorts sowie aus einer Komponente um Buchungen darzustellen und zu filtern. Ausserdem kann der Raum über den Reservieren-Button oben rechts reserviert werden.

Die Raum-Id ist in der ganzen Anwendung nicht ersichtlich, da sie für den Benutzer nicht relevant ist.

Für das Einbinden der OpenStreetMap habe ich, wie in der Aufgabenstellung vorgeschlagen, Leaflet verwendet. Für das Einstellen des Betrachtungszeitraums habe ich mich für eine externe DatePicker-Komponenten entschieden. Der Vorteil meiner Komponente liegt darin, dass der Benutzer *Von* und *Bis* in nur DatePicker-Dialog selektieren kann. Das heisst, der Benutzer spart sich Klicks beim Selektieren des Zeitraums und das UI wird durch das Wegfallen eines Inputfelds übersichtlicher. Das Handling der Daten mit Javascript ist eine Katastrophe und nicht wirklich praktikabel. Glücklicherweise habe ich mir mit der DatePicker-Komponente auch *Moment.js* ins Projekt geholt. *Moment.js* vereinfacht das Arbeiten und Umformattieren der Daten um ein Vielfaches.



Mockup Buchungsübersicht Mobile

Betrachtungszeitraum		24. December 2020 – 02. February 2021
Organisator Von	E-Mail Bis	Titel
Jonas 03. January 2021 19:00	jonas.vogel@ost.ch 03. January 2021 22:00	
Jonas 04. January 2021 13:15	jonas.vogel@ost.ch 04. January 2021 15:00	SWI (Best Modul eva)
Jonas 06. January 2021 14:00	jonas.vogel@ost.ch 06. January 2021 14:30	Präsentation BATH
Jonas 28. January 2021 19:00	jonas.vogel@ost.ch 28. January 2021 20:00	Coaching PPR3

Screenshot Buchungsübersicht Mobile

Die Schwierigkeit der Raumübersicht war die Responsiveness. Während die Rauminformationen und der Raumstandort auch auf kleinen Bildschirmen einfach benutzbar sind und sich einfach untereinander darstellen lassen, besteht beim Darstellen der Buchungen die Gefahr, dass Informationen abgeschnitten werden. Beispielsweise wäre die Funktionalität nicht gegeben, falls das Von- und Bis-Datum nicht vollständig dargestellt würde. Deshalb stelle ich die Daten in einem Listenelement dar, das auf kleinen Bildschirmen eine zweite Reihe beansprucht. Zulasten der Übersichtlichkeit kann ich so vollständige Funktionalität gewährleisten.

Beim Löschen-Button wird auf kleinen Bildschirmen die Beschriftung nicht angezeigt. Die Beschriftung hätte zwar Platz, allerdings wirkt das UI ohne die Beschriftung des Buttons auf kleinen Bildschirmen aufgeräumter und leichter.

Durch Klicken auf den Reservierungs-Button oben rechts gelangt der Benutzer zur Reservierung.

Reservierung

Raumreservierung
[Übersicht](#) > [Sitzungszimmer 1](#) > Reservieren

Name

E-Mail

Titel (Optional)

Dauer

CHF EUR
Kurs vom 11. Januar

Reservieren

Mockup Buchungsformular

Raumreservierung
[Übersicht](#) / [Seminarraum 49](#) / Reservieren

Name

E-Mail

Meetingtitel (Optional)

Dauer

Preis CHF Preis EUR

Reservieren

Screenshot Buchungsformular

Die Reservierungsseite enthält ein minimalistisches Reservierungsformular. Das Formular habe ich bewusst in einer Seite und nicht in einem Dialog eingebettet. Beim Dialog handelt es sich nicht um die im Web bevorzugte Lösung.

Name und Email-Adresse des Benutzers werden im *LocalStorage* gespeichert. Auch im Formular konnte ich durch meinen Datepicker ein Input-Feld sparen. Pflichtfelder werden nicht als solche gekennzeichnet. Kennzeichnet wird nur das Feld *Meetingtitel*, da es optional ist und somit vom Standard abweicht. Das Formular ist zentriert und nimmt nicht den gesamten zur Verfügung gestellten Platz ein. Es ist wenig sinnvoll, das Formular auf grossen Bildschirmen maximal wachsen zu lassen, da der Benutzer in diesem Fällen nicht das gesamte Formular im Blickwinkel hätte.

Der Preis der Reservierung wird in Readonly-Inputs dargestellt. Das ist unschön, da kein Inputfeld sondern nur ein Label benötigt würde. Allerdings müsste das Label noch manuell gestylt werden. Auf diesen Schritt habe ich bei der Entwicklung verzichtet. Hier ist sicherlich noch Optimierungspotential vorhanden.

Fehler, die beim Absenden der Reservierung auftreten, werden direkt auf dem Feld (wenn ein Pflichtfeld nicht ausgefüllt wurde) oder zuoberst vom Formular in einem Bootstrap-Alert (z.B. wenn der Server eine Zeitraum-Überschneidung meldet) dargestellt. Dies ermöglicht, dass der Benutzer gleichzeitig die Fehlermeldung einsehen und seine Eingaben korrigieren kann.

Nach erfolgreicher Reservierung wird der Benutzer auf die Raumübersicht navigiert, wo er die getätigte Buchung einsehen kann. Zusätzlich wird eine Erfolgsmeldung zuoberst auf der Seite dargestellt.

Entwicklung

Die Entwicklung des Raumreservierungssystems hat sich als schwieriger herausgestellt als erwartet.

In der Webentwicklung wird normalerweise ein Router eingesetzt. Der Router handelt die Navigation zwischen unterschiedlichen Seiten. So kann eine optimale Kapselung der Seiten erreicht werden. In diesem Projekt kann allerdings kein Router eingesetzt werden, da das Projekt lokal laufen muss.

Da ich es unschön finde, wenn nicht benötigter HTML-Code einfach ausgeblendet (`visible = false`) wird, aber für den Benutzer einsehbar wäre, habe ich mich dafür entschieden die gesamte App dynamisch aus dem Javascript-Code zu generieren.

Architektur

Der Einfachheit halber, habe ich alle externen Dependencies, ausser Bootstrap, über das Internet eingebunden. Das Bootstrap-Theme habe ich lokal eingebunden.

Ebenfalls aufgrund der Tatsache, dass der Code lokal laufen muss, können keine Module eingesetzt werden. Folglich habe ich versucht mit verschiedenen Files und Klassen eine gewissen Kapselung zu erreichen.

File	Zweck
<i>bookingform.js</i>	Dieses File beinhaltet den Code zum Bauen des Buchungs-Formulars. Für eine bessere Lesbarkeit habe ich einen <i>InputBuilder</i> geschrieben, mit welchem Input-Element durch ein einfaches Method-Chaining konfiguriert werden können.
<i>componentbuilders.js</i>	Das Builder-Pattern habe ich für verschiedene Komponenten verwendet. Der Builder-Code ist in das File <i>componentbuilders.js</i> ausgelagert. Beispielsweise werden die Karten oder die Breadcrumb über einen Builder erzeugt.
<i>componentfactory.js</i>	Diese Factory generiert HTML-Elemente. Der Code in dieser Klasse wurde durch das Buchungspanel in der Raumübersicht etwas aufgeblasen und unübersichtlich. Optimierungspotential bestünde darin, die Logik für das Buchungspanel herauszulösen und in eine eigene Klasse oder in ein eigenes File zu verschieben.
<i>constants.js</i>	Dieses File enthält Konstanten und zentrale Texte.

<i>pagehandler.js</i>	Der Pagehandler ist die oberste Instanz des Programmcodes und übernimmt die Aufgabe des Routers. Im Pagehandler ist definiert, welcher HTML-Code in den body gesetzt werden. Dieses File enthält ausserdem den <i>ErrorHandler</i> und den <i>ListenerStorage</i> . Der <i>ErrorHandler</i> steuert das Anzeigen der Bootstrap-Alerts, die zuoberst auf der Raum- oder Buchungs-Seite angezeigt werden. Mit dem <i>ListenerStorage</i> wird sichergestellt, dass alle Listener erst nach dem Erstellen des DOM-Trees gesetzt werden (siehe mehr dazu im Kapitel Probleme).
<i>restclient.js</i>	Über den <i>RestClient</i> werden jeglich Rest-Calls abgewickelt.
<i>util.js</i>	Dieses File enthält Utility-Code, welcher für die Einbindung und Konfiguration der externen Komponenten (Datepicker und Map) verwendet wird.

Codestyle

Javascript bietet bedauerlicherweise wenig Features, die wartbaren Code fördern. Um Seiteneffekten vorzubeugen, habe ich falls möglich immer das Keyword *const* zur Initialisierung der Variablen verwendet. Ausserdem habe ich die Namen von Methoden, die ausschliesslich für den internen Gebrauch in der Klasse (*private*) gedacht sind, mit einem Underline begonnen. Zudem achtete ich darauf, dass die Methoden kompakt sind. Wie bereits erwähnt habe ich das Builder-Pattern wiederholt eingesetzt, um den Code lesbarer zu machen und die Komponenten-Logik besser abzuschotten. Auf den Einsatz von JQuery habe ich verzichtet.

Probleme

Ich hatte bei meinem Projekt das Problem, dass ich keine Listener an meine dynamisch erzeugten Elemente anhängen konnte. Eine mögliche Ursache des Problems wäre, dass der DOM-Tree zum Zeitpunkt an dem die Listener angehängt werden, noch nicht aufgebaut ist. Um das Problem zu umgehen habe ich die Klasse *ListenerStorage* erstellt. Alle Listener werden dem *ListenerStorage* übergeben, welcher sicherstellt, dass alle Listener nach dem Darstellen der HTML-Komponenten an die entsprechenden Komponenten angehängt werden. Der *ListenerStorage* ist nichts was man in einem grösseren Projekt als wartbar bezeichnen würde.

Stärken

Die Stärke meines Raumreservierungssystems ist, dass der komplette Funktionsumfang in einem modernen und doch sehr praktischen Format daherkommt. Das Design ist schlicht und auf das Responsive-Verhalten habe ich durchaus viel Wert gelegt. Ausserdem kann der Benutzer, meiner Meinung nach, bei der Bedienung nicht viel falsch machen. Das Programm ist intuitiv und bietet keine Überraschungen.

Schwächen

Mein Raumreservierungssystem setzt voraus, dass der Benutzer weiss welchen Raum er buchen möchte. Aktuell ist es nicht möglich basierend auf einem Betrachtungszeitraum alle verfügbaren Räume anzuzeigen. Ausserdem wird keine horizontale Navigation zwischen den Räumen unterstützt, d.h. der einzige Weg von der einen Raumübersicht in die Übersicht eines anderen Raumes ist das Hauptmenü. Ein weiterer Schwachpunkt ist sicherlich, dass die Architektur aufgrund des *ListenerStorage* schlecht wartbar ist. Ausserdem ist meine Applikation anfällig für Cross-Side-Scripting-Schwachstellen. Da das Programm nur lokal läuft, stellt das aber kein kritisches Problem dar.

Reflexion

Ich habe mich anfangs mit Javascript sehr schwer getan. Die Entwicklung, vor allem das Debugging, war sehr mühsam und ich bevorzuge das Arbeiten mit mehr Restriktionen und Typensicherheit. Mit der Zeit fühlte ich mich allerdings immer sicherer.

Trotzdem würde ich im Nachhinein einiges anders machen. Der Architekturentscheid, den gesamten HTML-Code zu generieren war falsch. Ein besserer Lösungsansatz wäre die Arbeit mit mehreren HTML-Files. Für den Inhalt jeder Seite würde das Grundgerüst als eigenes HTML-Dokument geführt. Bei Navigation in der Anwendung müsste dann nur noch der jeweilige Seiteninhalt mit einem iFrame-Element eingebettet und um die dynamischen Elemente ergänzt werden. Eine alternative Herangehensweise wäre das Erstellen von drei unabhängige HTML-Seiten. Bei Navigation zwischen den Seiten hätten die Parameter (z.B. die Id des selektierten Raumes) in den Local- oder Session-Storage geschrieben werden können. Ich schätze, die potentielle Zeitersparnis auf 50 Prozent, wenn ich mehr statischen Code in HTML-Templates ausgelagert hätte. Ausserdem wäre die Codebasis überschaubarer und somit weniger fehleranfällig.

Alles in allem ist das Endprodukt nicht perfekt, doch ich bin sehr zufrieden damit. Die Implementierung war herausfordernd und hat mir viel Spass bereitet.